

TJSC 2026 - Analista de Sistemas

Engenharia de Software e Desenvolvimento

Aula 2 - Conceitos de Engenharia de Software e Processos de Desenvolvimento

Material autoral, orientado ao edital retificado do TJSC 2026 e calibrado por provas oficiais recentes da FGV. Foco em teoria objetiva, distinções de prova, exemplos corporativos e 20 questões inéditas comentadas.

Disciplina	Engenharia de Software e Desenvolvimento
Peso no edital	10 questões
Aula	2 de 17
Foco	Conceitos de engenharia de software e processos de desenvolvimento de software
Não aprofunda nesta aula	Scrum, Kanban, RUP, CMMI-DEV, MR-MPS-SW, testes, CI/CD e MoReq-Jus - temas tratados nas aulas próprias

1. Protocolo de escopo da aula

Esta aula cobre exatamente a Aula 2 da trilha atualizada: conceitos de engenharia de software e processos de desenvolvimento de software. O objetivo é formar a base conceitual para as aulas seguintes, sem antecipar em profundidade os tópicos que ganharão aula própria.

Fonte de escopo	Cobertura nesta aula
1. Conceitos de engenharia de software	Definição, finalidade, produto de software, processo de software, qualidade, manutenção, evolução, diferença entre software, sistema e aplicação.
1.1 Processos de desenvolvimento de software	Atividades fundamentais, ciclo de vida, modelos de processo, waterfall/cascata, V-Model, incremental, iterativo/evolutivo, prototipação, espiral, desenvolvimento baseado em componentes/reuso, escolha do processo por contexto.
Tópicos não aprofundados agora	Scrum/Kanban, RUP, CMMI-DEV v2.0, MR-MPS-SW 2024, testes unitários/integração/sistema, CI/CD e MoReq-Jus serão abordados depois para não misturar camadas do edital.

O Manual Mestre de Calibração FGV foi usado como régua editorial: a aula precisa preservar o edital, explicar do básico ao ponto de prova, usar exemplos técnicos compatíveis e elaborar questões com distratores plausíveis, formatos variados e comentários completos.

Índice sintético

Bloco	Conteúdo	Por que importa na FGV
1	Conceito de engenharia de software	A banca cobra distinções, não só definição decorada.
2	Produto, processo, projeto, sistema e aplicação	Termos parecidos aparecem como distratores.
3	Atividades fundamentais do processo	Base para requisitos, projeto, testes, manutenção e CI/CD.
4	Modelos de processo	Waterfall, V-Model, incremental, iterativo, prototipação, espiral e reuso caem por comparação.
5	Escolha do processo	Cenário institucional + restrições + risco = tipo de processo adequado.
6	Questões FGV autorais	20 itens com comentários completos.

2. Como a FGV vem cobrando o assunto

Em provas recentes de Tecnologia da Informação, a FGV costuma contextualizar o item em uma instituição pública, projeto corporativo, sistema de documentos, portal ou equipe de desenvolvimento. O comando raramente fica no nível 'defina X'; normalmente pede a classificação correta de uma situação, a análise de afirmativas, a identificação de uma fase do processo ou a escolha de um modelo de desenvolvimento compatível com restrições do caso.

Na calibração desta aula, foram observados cadernos oficiais da FGV com questões sobre metodologias prescritivas e ágeis, SWEBOK - Software Engineering Body of Knowledge, requisitos, V/F e afirmativas I, II e III. Isso sugere três padrões úteis para esta aula:

- a FGV gosta de comparar termos vizinhos: processo, metodologia, modelo, framework, ciclo de vida e prática;
- a FGV valoriza exemplos de contexto: órgão público, sistema de protocolo, módulo de apoio, API, portal, integração e manutenção;
- a FGV usa alternativas que parecem corretas em outro contexto, mas erram por escopo, temporalidade, finalidade ou excesso de generalização.

Nesta aula, portanto, as questões não foram feitas com enunciados telegráficos. A intenção é treinar o mesmo músculo que a banca exige: reconhecer o conceito correto dentro de uma situação parecida com a realidade de

desenvolvimento, sem ser enganado por palavras verdadeiras em lugar errado.

3. Engenharia de software: conceito e finalidade

Engenharia de software **é a aplicação sistemática**, disciplinada e quantificável de princípios, métodos, processos, técnicas e ferramentas para desenvolver, operar, manter e evoluir software. Em prova, essa definição aparece com várias faces, mas a ideia central é estável: software não deve ser tratado como improviso de codificação; ele deve ser construído por um processo controlável, com requisitos, projeto, implementação, verificação, validação, manutenção e gestão.

No seu trabalho com Java/Quarkus, Angular/TypeScript, APIs e esteiras corporativas, isso aparece de forma prática: antes de codificar um endpoint, alguém precisa entender a regra de negócio; antes de expor a API, alguém define contrato; antes de implantar, alguém testa; depois de subir produção, alguém monitora, corrige e evolui. A FGV, porém, não cobra sua vivência como relato. Ela cobra a terminologia formal dessas etapas.

Termo	Ideia de prova	Exemplo corporativo
Software	Conjunto de programas, dados, documentação e configurações associadas à solução.	Backend Quarkus, frontend Angular, migrations, documentação da API e parâmetros de ambiente.
Sistema	Conjunto maior que pode incluir software, pessoas, processos, hardware, dados e integrações.	Sistema de pagamentos integrado a autorização, contas, Pix, mensageria, logs e atendimento.
Produto de software	Resultado entregue para uso, interno ou externo, com funcionalidades e atributos de qualidade.	Portal, serviço REST, app web, módulo de gestão documental.
Processo de software	Conjunto estruturado de atividades usadas para produzir e evoluir software.	Fluxo de requisitos -> desenho -> codificação -> teste -> homologação -> deploy -> manutenção.
Projeto de software	Esforço temporário para criar ou modificar um produto de software.	Entrega de uma nova funcionalidade, integração ou módulo.

Pegadinha FGV: engenharia de software não é sinônimo de programação. Programar é uma atividade importante, mas a engenharia de software envolve também requisitos, arquitetura, projeto, qualidade, processo, gestão, manutenção, documentação e evolução.

3.1 Software é desenvolvido, não fabricado em série

Um produto físico costuma ter custo relevante de fabricação por unidade. Software tem outro comportamento: o custo principal está em concepção, desenvolvimento, testes, manutenção e evolução. Copiar o artefato executável é barato; mantê-lo correto, seguro, integrado e útil ao longo do tempo é caro. Por isso, a manutenção e a evolução são tão importantes em Engenharia de Software.

Na prova, se aparecer a frase 'software não se desgasta como hardware', a ideia é que ele não sofre desgaste físico. Entretanto, software se degrada por mudanças mal controladas, dependências obsoletas, acúmulo de dívida técnica, alterações de regra de negócio e ambiente operacional mutável.

3.2 Atributos de um bom software

Não basta o software 'rodar'. Em Engenharia de Software, qualidade envolve atributos como **manutenibilidade**, **confiabilidade**, **usabilidade**, **eficiência**, **segurança**, **portabilidade** e **adequação funcional**. Esses atributos serão aprofundados em aulas de qualidade e testes, mas aqui você precisa saber que eles orientam processos e decisões de desenvolvimento.

Atributo	Pergunta central	Erro comum em prova
Manutenibilidade	É fácil corrigir, adaptar, evoluir e entender?	Confundir com disponibilidade. Um sistema pode estar disponível e ser difícil de manter.
Confiabilidade	O sistema executa corretamente por um período e sob condições especificadas?	Tratar como sinônimo de segurança da informação.

Atributo	Pergunta central	Erro comum em prova
Usabilidade	O usuário consegue aprender e usar com efetividade?	Reduzir a layout bonito.
Eficiência/desempenho	Usa recursos e tempo de resposta adequadamente?	Confundir sempre com escalabilidade.
Segurança	Protege dados e operações contra acessos, alterações e uso indevido?	Achar que é só criptografia.
Portabilidade	Pode ser adaptado a outros ambientes?	Confundir com interoperabilidade entre sistemas.

4. Processo de software

Processo de software é um conjunto de atividades, papéis, artefatos, métodos e controles usados para desenvolver, entregar, manter e evoluir software. O processo não é o software; é o **caminho organizado para produzi-lo e modificá-lo**.

Um processo pode ser leve ou pesado, preditivo ou adaptativo, formal ou enxuto, iterativo ou sequencial. O ponto que a FGV explora é a adequação: modelos diferentes servem melhor para riscos, requisitos, prazos e níveis de mudança diferentes.

Conceito	Definição curta	Não confundir com
Processo	Como o trabalho de software é organizado.	Produto final.
Método	Técnica ou abordagem para executar uma atividade.	Ferramenta.
Ferramenta	Software de apoio ao processo.	Processo em si.
Modelo de processo	Representação abstrata de como o processo se organiza ao longo do ciclo de vida.	Norma de maturidade.
Metodologia	Conjunto mais prescritivo de práticas, papéis, artefatos e regras.	Uma prática isolada.
Framework	Estrutura adaptável de princípios, papéis ou práticas.	Receita detalhada e invariável.

4.1 Atividades fundamentais do processo

Uma forma clássica de enxergar processos de software é separar quatro macroatividades: **especificação**, **desenvolvimento**, **validação** e **evolução**. Elas aparecem com nomes variáveis em livros, normas, modelos e bancas, mas a lógica é recorrente.

Macroatividade	O que resolve	Exemplo em sistema corporativo
Especificação	Entender e definir o que o software deve fazer e sob quais restrições.	Definir regras de aprovação, contrato de API, perfis de acesso e requisitos não funcionais.
Desenvolvimento	Projetar e construir a solução.	Modelar entidades, criar endpoints, implementar frontend, persistência e integrações.
Validação	Verificar se o software atende às necessidades dos usuários e requisitos definidos.	Teste de aceite, homologação, revisão de requisitos, testes de sistema.
Evolução	Modificar o software em resposta a mudanças de negócio, ambiente ou defeitos.	Ajustar regra por mudança normativa, trocar integração, atualizar dependência, corrigir bug.

Pegadinha FGV: verificação e validação não são sinônimos perfeitos. Verificação pergunta se o produto foi construído corretamente em relação a especificações; validação pergunta se o produto certo foi construído para atender à necessidade real. A banca pode usar essa distinção para derrubar candidato que sabe a prática, mas não domina o vocabulário.

4.2 Ciclo de vida de software

Ciclo de vida de software é a **visão das fases** pelas quais um produto passa **desde a ideia até sua retirada, substituição ou descontinuação**. Em concursos, as fases costumam aparecer como: **planejamento, requisitos, análise, projeto, implementação, testes, implantação, operação, manutenção e evolução**. A ordem e o nome mudam conforme o modelo, mas a ideia é organizar o trabalho ao longo do tempo.

Em um ambiente de esteira corporativa, o ciclo de vida não termina no merge da branch. Ele inclui **observabilidade, correção, incidentes, atualização tecnológica, adequação normativa e evolução funcional**. Para a prova, guarde: desenvolvimento de software moderno é ciclo contínuo, mas os modelos prescritivos ainda são cobrados como base conceitual.

Fluxo genérico de ciclo de vida

Ideia ou demanda

- > estudo de viabilidade / planejamento
- > requisitos
- > análise e projeto
- > implementação
- > testes e validação
- > implantação / entrega
- > operação, manutenção e evolução
- > substituição ou descontinuação

5. Modelos de processo de desenvolvimento

Modelo de processo é uma representação simplificada de como as atividades de desenvolvimento são organizadas. Ele não descreve cada detalhe do trabalho, mas orienta a sequência, a iteração, o controle e a resposta a mudanças. A FGV costuma cobrar o modelo adequado ao cenário ou a característica que diferencia um modelo de outro.

Atenção: Scrum, Kanban e RUP aparecem no edital e terão aulas próprias. Aqui eles são citados apenas para posicionamento no mapa geral de processos.

5.1 Modelo em cascata - Waterfall

O modelo em cascata organiza o desenvolvimento em fases sequenciais, com **forte ênfase em planejamento, documentação e conclusão de uma etapa antes da próxima**. A imagem mental é uma queda d'água: requisitos -> projeto -> implementação -> testes -> implantação -> manutenção.

Ponto	Como cai
Força	Boa previsibilidade quando requisitos são estáveis, domínio conhecido e mudanças são pouco prováveis .
Fraqueza	Pouca flexibilidade diante de mudanças tardias ; feedback real do usuário pode chegar tarde.
Contexto típico	Projetos regulados, contratos fechados, escopo estável, necessidade forte de documentação.
Pegadinha	Não significa ausência total de retorno entre fases; em prova, o traço distintivo é a sequência planejada e pouco adaptativa.

5.2 Modelo em V - V-Model

O V-Model explicita a relação entre **atividades de desenvolvimento e níveis de teste**. Um lado do V representa especificação e projeto; o outro lado representa verificação e validação correspondentes. A ideia de prova é a correspondência: requisitos de usuário se relacionam a testes de aceitação; requisitos de sistema a testes de sistema; projeto arquitetural a testes de integração; projeto detalhado/código a testes unitários.

Lado de especificação/projeto	Lado de teste correspondente
Requisitos de usuário	Teste de aceitação
Requisitos de sistema	Teste de sistema
Projeto arquitetural	Teste de integração
Projeto detalhado / unidades	Teste unitário

Pegadinha FGV: o V-Model não é simplesmente 'cascata com desenho bonito'. Seu diferencial é evidenciar a associação entre níveis de especificação/projeto e níveis de teste.

5.3 Modelo incremental

No modelo incremental, o sistema é entregue em partes funcionais, chamadas incrementos. Cada incremento **adiciona capacidade ao produto**. O usuário pode começar a obter valor antes da entrega completa do sistema.

Exemplo: um sistema de gestão documental pode primeiro entregar cadastro e consulta; depois assinatura; depois tramitação; depois preservação; depois relatórios. Cada incremento deve agregar funcionalidade utilizável, e não apenas uma camada técnica invisível sem valor de negócio.

Vantagem	Risco/limite	Pegadinha
Entrega valor mais cedo e reduz risco de esperar o sistema completo.	Exige arquitetura que permita crescimento sem remendos.	Incremental não é igual a protótipo descartável. Incremento é parte real do produto.

5.4 Modelo iterativo e evolutivo

No desenvolvimento iterativo, o time revisita atividades em ciclos, refinando requisitos, projeto e implementação. O produto nasce em versões sucessivas e melhora por feedback. No desenvolvimento evolutivo, a solução é construída e ajustada à medida que o entendimento do problema amadurece.

A diferença prática para a prova: incremento enfatiza adicionar partes funcionais; iteração enfatiza repetir ciclos de refinamento. Eles frequentemente coexistem, mas não são sinônimos obrigatórios.

Termo	Ideia central	Exemplo
Incremental	Entregar pedaços funcionais sucessivos.	Módulo de cadastro, depois protocolo, depois tramitação.
Iterativo	Refinar uma solução por ciclos.	Revisar interface e regras após feedback de usuários.
Evolutivo	Permitir que o produto amadureça com mudanças e aprendizado.	Ajustar o sistema conforme mudança normativa e uso real.

5.5 Prototipação

Prototipação é a construção de uma versão parcial, simplificada ou experimental para explorar requisitos, interface, viabilidade técnica ou entendimento do domínio. Pode ser descartável ou evolutiva.

Tipo	Característica	Erro comum
Protótipo descartável	Usado para aprender e depois descartado.	Transformar o protótipo frágil em produção sem reengenharia.
Protótipo evolutivo	É refinado até virar produto ou parte dele.	Achar que todo protótipo deve ser jogado fora.
Prova de conceito - PoC	Testa viabilidade técnica de uma ideia.	Confundir com produto completo validado.

Em Angular, uma tela navegável com dados mockados pode esclarecer fluxo do usuário. Em Quarkus, um endpoint simples pode validar integração com serviço externo. Para a FGV, o ponto é: protótipo reduz incerteza e melhora comunicação com stakeholders, mas não substitui especificação, arquitetura, qualidade e testes quando o produto virar produção.

5.6 Modelo espiral

O **modelo espiral** organiza o desenvolvimento em **ciclos guiados por análise de riscos**. Cada **volta da espiral envolve planejamento, análise de riscos, engenharia/desenvolvimento e avaliação**. Ele é associado a projetos grandes, complexos ou com riscos significativos.

Elemento-chave	Como reconhecer na prova
Risco	Se o enunciado enfatiza análise e redução de risco a cada ciclo , pense em espiral.
Ciclos	O desenvolvimento avança em voltas sucessivas, não em uma sequência linear única.
Adequação	Projetos complexos, críticos, caros ou tecnologicamente incertos.
Pegadinha	Não é simplesmente incremental. O diferencial é a gestão explícita de riscos.

5.7 Desenvolvimento baseado em componentes e reuso

O desenvolvimento baseado em componentes ou orientado a reuso parte da ideia de construir sistemas aproveitando componentes, bibliotecas, serviços, frameworks, APIs e módulos já existentes. Em vez de desenvolver tudo do zero, a equipe integra e adapta elementos reutilizáveis.

No seu contexto, isso aparece quando você usa bibliotecas de autenticação, componentes do design system, clientes HTTP padronizados, APIs corporativas, templates de esteira e dependências Maven/npm. Para prova, porém, a banca quer a consequência conceitual: reuso tende a reduzir esforço e risco quando os componentes são adequados, mas pode introduzir dependência, restrições de integração, licenciamento, compatibilidade e acoplamento externo.

Vantagem	Risco
Menor tempo de desenvolvimento se os componentes forem confiáveis.	Dependência de fornecedor, versão, contrato, API ou componente.
Padronização e reaproveitamento institucional.	Componente inadequado pode forçar arquitetura ruim.
Menor retrabalho em funcionalidades comuns.	Necessidade de testar integração e compatibilidade.

6. Comparativo de modelos de processo

Modelo	Essência	Quando tende a servir	Armadilha FGV
Cascata / Waterfall	Fases sequenciais e planejamento forte.	Escopo estável, requisitos bem conhecidos, documentação forte.	Achar que nunca há feedback ou mudança.
V-Model	Correspondência entre especificação/projeto e testes.	Sistemas que exigem rastreabilidade e verificação por níveis.	Reduzir a 'cascata com testes no fim'.
Incremental	Entregas funcionais sucessivas.	Quando é útil entregar valor parcial cedo.	Confundir incremento real com protótipo descartável.
Iterativo/evolutivo	Ciclos de refinamento e aprendizado.	Quando requisitos amadurecem com feedback.	Tratar iteração e incremento como sempre idênticos.
Prototipação	Aprendizado por versão parcial ou experimental.	Quando há incerteza de requisitos, interface ou viabilidade.	Levar protótipo frágil para produção sem cuidado.
Espiral	Ciclos orientados por riscos.	Projetos grandes, críticos, caros ou incertos.	Esquecer que risco é o elemento distintivo.
Baseado em componentes/reuso	Integra elementos existentes.	Quando há componentes confiáveis e compatíveis.	Supor que reuso elimina teste e adaptação.

7. Preditivo, adaptativo e híbrido

A FGV também gosta da divisão ampla entre abordagens prescritivas/preditivas e abordagens adaptativas/ágeis. Como Scrum e Kanban terão aula própria, aqui a distinção deve ser entendida em alto nível.

Abordagem	Características	Melhor sinal no enunciado
Preditiva/prescritiva	Planejamento mais forte no início, escopo mais controlado, maior formalidade documental, fases mais definidas.	Requisitos estáveis, contrato rígido, alta necessidade de documentação, previsibilidade.
Adaptativa/ágil	Iteração curta, resposta a mudanças, feedback contínuo, colaboração com cliente, entrega frequente de valor.	Requisitos incertos, ambiente mutável, necessidade de feedback rápido.

Abordagem	Características	Melhor sinal no enunciado
Híbrida	Combina governança e previsibilidade com ciclos iterativos e práticas ágeis.	Órgão público com controle formal, mas entregas parciais e validação frequente.

Pegadinha FGV: abordagem ágil não significa ausência de documentação, arquitetura, teste ou planejamento. Significa responder melhor a mudanças e entregar valor de forma iterativa, com documentação suficiente e útil. Do outro lado, modelo preditivo não significa processo ruim; significa maior adequação quando o escopo é estável e a previsibilidade é requisito relevante.

8. Como escolher o processo em uma questão

Quando a FGV monta um cenário, leia primeiro os sinais. Não responda pelo método que você prefere no trabalho. Responda pelo processo que melhor se ajusta ao conjunto de restrições apresentado.

Sinal no enunciado	Processo provável	Cuidado
Requisitos estáveis, escopo contratual, baixa mudança	Cascata ou preditivo	Não escolher ágil só por ser moderno.
Rastreabilidade forte entre requisitos e testes	V-Model	O diferencial é a associação com testes.
Entrega de valor por partes utilizáveis	Incremental	A parte entregue deve funcionar como incremento.
Aprendizado e refinamento por ciclos	Iterativo/evolutivo	Pode coexistir com incremental.
Dúvida sobre interface, requisito ou viabilidade	Prototipação	Pode ser descartável ou evolutiva.
Projeto crítico, caro, incerto, com riscos relevantes	Espiral	Foco explícito em risco.
Aproveitamento de componentes, bibliotecas, APIs, frameworks	Baseado em reuso/componentes	Reuso não elimina integração e testes.

9. Diferença entre uso real e cobrança FGV

No trabalho real, equipes misturam práticas: um órgão pode exigir documentação formal, usar Git, esteira CI/CD, deploy automatizado, reuniões ágeis e aprovação por comitê. A prova, porém, costuma isolar um traço conceitual. Se o enunciado enfatiza risco, pense em espiral; se enfatiza testes correspondentes a especificações, pense em V-Model; se enfatiza entrega em partes, pense em incremental.

No trabalho	Na prova
O time usa Jira, Git, pipeline, documentação e cerimônias ao mesmo tempo.	A questão isola o conceito que melhor explica a situação.
Um processo pode ser híbrido e adaptado.	A banca pode pedir o modelo clássico mais associado a um sinal do enunciado.
Protótipo pode virar produto por pressão de prazo.	Conceitualmente, protótipo descartável não deveria ir a produção sem reengenharia.
Cascata pode ter ajustes e revisões.	Em prova, continua sendo modelo sequencial e preditivo.
Ágil não dispensa documentação útil.	Alternativa que diz 'ágil não documenta' geralmente exagera e tende a estar errada.

10. Pegadinhas FGV desta aula

Engenharia de software não é sinônimo de programação; programação é parte da engenharia de software.

Processo de software não é produto de software; processo é o caminho, produto é o resultado.

Ciclo de vida não é metodologia específica; é a visão das fases do software ao longo do tempo.

Modelo incremental entrega funcionalidades reais em partes; protótipo pode ser apenas instrumento de aprendizado.

Iterativo e incremental costumam andar juntos, mas enfatizam coisas diferentes: repetir/refinar versus entregar partes.

Espiral é orientado por riscos; se não houver sinal de risco, talvez não seja a melhor resposta.

V-Model relaciona fases de especificação/projeto com níveis de teste; não é só 'testar no fim'.

Ágil não é bagunça; preditivo não é necessariamente ultrapassado. A adequação depende do contexto.

Reuso reduz esforço quando bem aplicado, mas não dispensa avaliação, integração e testes.

11. Questões autorais - padrão FGV

Resolva as questões antes de consultar o gabarito. As questões são inéditas e foram elaboradas para simular densidade, vocabulário e formato de cobrança da FGV, sem copiar itens reais.

Questão 1

Uma equipe de desenvolvimento de um tribunal recebeu a demanda de construir um módulo de consulta processual. O gerente percebeu que parte dos desenvolvedores tratava a atividade como mera escrita de código, sem registrar decisões de projeto, critérios de aceite, riscos de integração ou plano de testes. À luz dos conceitos de Engenharia de Software, a interpretação mais adequada é que a disciplina busca:

- (A) substituir a documentação por código executável, pois o software só existe quando compilado e implantado.
- (B) limitar o trabalho técnico à programação, já que requisitos e testes pertencem a áreas administrativas.
- (C) aplicar abordagem sistemática, disciplinada e controlável ao desenvolvimento, operação, manutenção e evolução do software.
- (D) restringir a criação de software a linguagens orientadas a objetos e bancos relacionais.
- (E) eliminar mudanças durante o desenvolvimento, pois todo software correto nasce de requisitos completos e imutáveis.

Questão 2

No planejamento de um sistema corporativo, o analista distinguiu produto de software, processo de software e projeto de software. Considerando essa distinção, assinale a afirmativa correta.

- (A) Processo de software é o executável implantado em produção, enquanto produto de software é a sequência de atividades para gerá-lo.
- (B) Projeto de software é um esforço temporário destinado a criar ou modificar um produto de software, dentro de restrições de escopo, prazo, custo e qualidade.
- (C) Produto de software se resume ao código-fonte, excluindo documentação, dados de configuração e artefatos de implantação.
- (D) Processo e metodologia são sinônimos estritos, ambos designando uma ferramenta CASE de automação.
- (E) Sistema e software são conceitos sempre equivalentes, pois ambos excluem pessoas, processos, dados e hardware.

Questão 3

Considere as afirmativas sobre características do software. I. Software não sofre desgaste físico como hardware, mas pode deteriorar-se em qualidade por mudanças mal controladas, dependências obsoletas e dívida técnica. II. O custo de software concentra-se apenas na duplicação das cópias distribuídas aos usuários finais. III. A manutenção e a evolução são dimensões relevantes porque regras de negócio, tecnologias, integrações e ambientes mudam ao longo do tempo. Está correto o que se afirma em

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) II e III, apenas.
- (E) I e III, apenas.

Questão 4

Uma secretaria pretende contratar o desenvolvimento de um sistema cujo escopo está bem definido por norma interna, com baixa expectativa de mudança, necessidade de documentação formal e marcos de aprovação ao final de cada fase. O modelo de processo mais compatível, em termos clássicos, é o

- (A) prototipação descartável, pois todo sistema normativo deve começar por uma interface incompleta.
- (B) espiral, pois qualquer contratação pública é necessariamente orientada por riscos tecnológicos extremos.
- (C) baseado em componentes, pois documentação formal impede desenvolvimento sequencial.
- (D) cascata, por enfatizar fases sequenciais, planejamento e maior previsibilidade quando os requisitos são estáveis.
- (E) Kanban, porque todo fluxo com etapas de aprovação é automaticamente Kanban.

Questão 5

No contexto de processos de software, a diferença mais adequada entre ciclo de vida e modelo de processo é que o ciclo de vida

- (A) representa as fases pelas quais o software passa ao longo do tempo, enquanto o modelo de processo descreve uma forma abstrata de organizar atividades de desenvolvimento e evolução.
- (B) é uma linguagem de programação voltada à implementação de sistemas, enquanto o modelo de processo é uma biblioteca de componentes reutilizáveis.
- (C) é sempre ágil, enquanto o modelo de processo é sempre preditivo e documental.
- (D) corresponde exclusivamente à fase de codificação, enquanto o modelo de processo corresponde exclusivamente ao teste de sistema.
- (E) é o conjunto de ferramentas automatizadas de versionamento, enquanto o modelo de processo é o ambiente de execução.

Questão 6

Em uma iniciativa de desenvolvimento de software, a equipe precisa definir o que será construído, construir a solução, verificar se ela atende às necessidades esperadas e, depois, adaptar o sistema a mudanças de regra e ambiente. Em termos de macroatividades de processo, essa sequência corresponde, de modo mais adequado, a

- (A) implantação, contratação, auditoria e descarte.
- (B) programação, compilação, criptografia e hospedagem.
- (C) especificação, desenvolvimento, validação e evolução.
- (D) modelagem física, normalização, indexação e replicação.
- (E) roteamento, autenticação, autorização e contabilização.

Questão 7

Assinale a opção que melhor diferencia verificação e validação no contexto de Engenharia de Software.

- (A) Verificação é realizada apenas pelo usuário final; validação é realizada apenas pelo compilador.
- (B) Verificação analisa se o produto foi construído corretamente em relação a especificações; validação analisa se o produto certo foi construído para atender à necessidade real.
- (C) Verificação é sinônimo de teste de usabilidade; validação é sinônimo de controle de versão.
- (D) Verificação ocorre somente após a implantação em produção; validação ocorre somente antes dos requisitos.
- (E) Verificação e validação são termos obsoletos, substituídos integralmente por CI/CD.

Questão 8

Uma equipe deseja reforçar a rastreabilidade entre artefatos de especificação e níveis de teste. O analista propõe um modelo em que requisitos de usuário se relacionam a testes de aceitação, requisitos de sistema a testes de sistema, arquitetura a testes de integração e unidades de software a testes unitários. O modelo descrito é o

- (A) cascata puro, sem qualquer relação explícita com testes.
- (B) espiral, pois todo teste deriva de risco financeiro.
- (C) incremental, pois cada teste corresponde a uma entrega parcial.
- (D) prototipação rápida, pois o foco é simular telas.
- (E) V-Model, por evidenciar correspondência entre especificação/projeto e níveis de teste.

Questão 9

Um órgão público deseja substituir gradualmente um sistema legado. A primeira entrega deve disponibilizar cadastro e consulta; a segunda, tramitação; a terceira, relatórios; e a quarta, integração com serviços externos. Cada entrega deve ser utilizável e agregar valor ao negócio. O modelo descrito é melhor caracterizado como

- (A) modelo cascata sem entregas intermediárias.
- (B) prototipação descartável, pois cada parte deve ser descartada ao final.
- (C) modelo exclusivamente orientado a testes de unidade.
- (D) desenvolvimento incremental.
- (E) modelo de maturidade de processo.

Questão 10

Durante a construção de um portal, a equipe entrega sucessivas versões da mesma funcionalidade de pesquisa, refinando filtros, layout, desempenho e critérios de ordenação a partir de feedback dos usuários. O elemento mais diretamente caracterizado no cenário é

- (A) normalização de dados.
- (B) iteração de desenvolvimento.
- (C) controle de concorrência.
- (D) assinatura digital.
- (E) particionamento físico.

Questão 11

Sobre prototipação em processos de software, assinale a opção correta.

- (A) Protótipos podem ser usados para reduzir incerteza sobre requisitos, interface ou viabilidade, podendo ser descartáveis ou evolutivos.
- (B) Todo protótipo é produto final e deve ser implantado em produção sem alteração arquitetural.
- (C) Prototipação é incompatível com participação de usuários, pois seu objetivo é evitar feedback externo.
- (D) Protótipo descartável e incremento são sinônimos, pois ambos devem ser preservados como versão final.
- (E) Prototipação elimina a necessidade de validação, documentação mínima e testes.

Questão 12

Um projeto envolve integração com tecnologias pouco conhecidas, alto custo de falha, forte impacto institucional e necessidade de avaliar riscos técnicos a cada ciclo. O modelo de processo clássico mais associado a esse tipo de cenário é o

- (A) cascata linear, por eliminar a análise de riscos após a primeira fase.
- (B) modelo em V, por tratar riscos apenas como casos de teste unitário.
- (C) espiral, por organizar ciclos com ênfase explícita na identificação e mitigação de riscos.
- (D) modelo Big Bang, por iniciar codificação sem planejamento formal.
- (E) modelo baseado em cópias físicas, por reduzir o custo marginal do software.

Questão 13

Relacione os modelos de processo às características apresentadas. 1. Cascata. 2. Incremental. 3. Espiral. 4. Prototipação. () Entrega partes funcionais sucessivas do produto. () Enfatiza fases sequenciais e planejamento inicial. () Usa ciclos orientados por análise e mitigação de riscos. () Constrói versão parcial para explorar requisitos, interface ou viabilidade. A sequência correta é

- (A) 1 - 2 - 3 - 4.
- (B) 2 - 1 - 4 - 3.
- (C) 4 - 1 - 3 - 2.
- (D) 3 - 4 - 2 - 1.
- (E) 2 - 1 - 3 - 4.

Questão 14

Assinale a alternativa que apresenta uma interpretação correta sobre desenvolvimento baseado em componentes e reuso.

- (A) O reuso elimina a necessidade de testes, pois componentes reutilizados já foram usados em outros sistemas.
- (B) O reuso impede integração com APIs e frameworks, por exigir implementação integral do zero.
- (C) O desenvolvimento baseado em componentes é restrito a linguagens funcionais e não se aplica a sistemas corporativos.
- (D) O reuso transforma automaticamente qualquer sistema em protótipo descartável.
- (E) O reuso pode reduzir esforço e risco quando os componentes são adequados, mas exige avaliação de compatibilidade, integração, dependências e qualidade.

Questão 15

Assinale a opção incorreta sobre processos de desenvolvimento de software.

- (A) Um processo pode ser adaptado ao contexto do projeto, desde que preserve controles necessários de qualidade e gestão.
- (B) Quanto mais moderno o processo, menor a necessidade de requisitos, testes e comunicação com stakeholders.
- (C) Modelos preditivos tendem a ser mais adequados quando requisitos são estáveis e há necessidade de previsibilidade.
- (D) Modelos adaptativos tendem a lidar melhor com mudanças frequentes e feedback contínuo.
- (E) Processos híbridos podem combinar governança formal com ciclos iterativos de entrega.

Questão 16

Analise as afirmativas. I. Abordagens preditivas favorecem maior planejamento inicial e controle de escopo quando há boa estabilidade de requisitos. II. Abordagens adaptativas favorecem feedback frequente, resposta a mudanças e entregas iterativas de valor. III. Em qualquer cenário, a adoção de uma abordagem ágil elimina a necessidade de documentação, arquitetura e testes formais. Está correto o que se afirma em

- (A) I, II e III.
- (B) I e III, apenas.
- (C) I e II, apenas.
- (D) II, apenas.
- (E) III, apenas.

Questão 17

Um gerente afirma que, como a equipe adotou práticas ágeis, o projeto não precisa mais de planejamento nem documentação. Sob a perspectiva correta de Engenharia de Software, essa afirmação é

- (A) equivocada, pois agilidade não elimina planejamento nem documentação útil; ela busca adaptação, feedback e entrega frequente de valor.
- (B) correta, pois toda abordagem ágil proíbe registro escrito de decisões técnicas.
- (C) correta, pois processos ágeis são incompatíveis com testes automatizados e arquitetura evolutiva.
- (D) equivocada apenas porque planejamento é tarefa exclusiva da área jurídica, e não da equipe de software.
- (E) correta apenas quando a equipe usa Java ou TypeScript.

Questão 18

Considere um órgão que precisa de forte documentação para auditoria, mas também deseja entregar funcionalidades em ciclos curtos, validando cada entrega com usuários internos. A abordagem mais coerente com esse cenário é

- (A) adotar exclusivamente o modelo Big Bang, pois documentação e validação são sinais de imprevisto.
- (B) impedir qualquer iteração, pois auditoria e entregas parciais são incompatíveis.
- (C) classificar automaticamente o projeto como prototipação descartável.
- (D) substituir todo o processo por uma ferramenta de versionamento.
- (E) usar uma abordagem híbrida, combinando governança formal com entregas iterativas e validação frequente.

Questão 19

Em uma análise de processo, o auditor identificou que a equipe possuía repositório Git, pipeline de build, ferramenta de issues e wiki, mas não havia critérios claros de passagem entre etapas, responsáveis definidos, artefatos mínimos nem política de validação. A conclusão mais adequada é que

- (A) a existência de ferramentas garante, por si só, um processo de software maduro e completo.
- (B) ferramentas CASE e de versionamento substituem atividades de requisitos, projeto e validação.
- (C) um processo de software é dispensável quando há automação de build.
- (D) ferramentas apoiam o processo, mas não se confundem com o processo nem substituem sua definição.
- (E) o uso de Git caracteriza obrigatoriamente o modelo em V.

Questão 20

Um analista está comparando dois modelos. No primeiro, o objetivo é entregar módulos funcionais em sequência, cada um agregando valor ao usuário. No segundo, o objetivo é repetir ciclos de refinamento sobre solução, requisitos e projeto, melhorando o produto a cada rodada. A distinção apresentada corresponde, respectivamente, a

- (A) prototipação descartável e cascata.
- (B) incremental e iterativo.
- (C) espiral e V-Model.
- (D) DevOps e waterfall.
- (E) modelo físico e modelo conceitual.

12. Gabarito resumido

1-C 2-B 3-E 4-D 5-A 6-C 7-B 8-E 9-D 10-B 11-A 12-C 13-E 14-E 15-B 16-C 17-A 18-E 19-D 20-B

Distribuição orgânica resultante: A = 3, B = 5, C = 4, D = 3, E = 5. A distribuição não foi forçada matematicamente; as alternativas foram embaralhadas por plausibilidade e auditadas para evitar padrão visual evidente.

13. Comentários completos

Questão 1 - Gabarito: C

- (A) Errada. Código executável é parte do produto, mas engenharia de software não substitui documentação e controle por código puro.
- (B) Errada. Requisitos e testes fazem parte do processo de software; não são atividades externas sem relação com engenharia.
- (C) Correta. A alternativa expressa a ideia central: abordagem sistemática e controlável para desenvolvimento, operação, manutenção e evolução.
- (D) Errada. Engenharia de software não se restringe a paradigma OO nem a bancos relacionais.
- (E) Errada. Mudanças podem e costumam ocorrer. Processos devem controlá-las, não presumir que nunca existam.

Erro provável: conceito.

Questão 2 - Gabarito: B

- (A) Errada. Inverte produto e processo.
- (B) Correta. Projeto é esforço temporário com objetivo definido, restrições e entrega de produto ou modificação.
- (C) Errada. Produto de software inclui mais que código-fonte.
- (D) Errada. Processo, metodologia e ferramenta não são sinônimos estritos.
- (E) Errada. Sistema é conceito mais amplo que software, podendo envolver pessoas, processos, hardware e dados.

Erro provável: confusão entre termos.

Questão 3 - Gabarito: E

- (A) Correta em conteúdo, mas não é a resposta completa porque a III também está correta.
- (B) Errada. Duplicação de cópias não é o custo principal do software; desenvolvimento e evolução concentram esforço.
- (C) Errada. Inclui a II, que está incorreta.
- (D) Errada. Inclui a II, que está incorreta, embora a III esteja correta.
- (E) Correta. I e III capturam duas propriedades clássicas: ausência de desgaste físico e necessidade de manutenção/evolução.

Erro provável: conceito.

Questão 4 - Gabarito: D

- (A) Errada. Prototipação ajuda em incertezas, não é obrigatória por ser sistema normativo.
- (B) Errada. Espiral é associado a riscos relevantes e incerteza, não a qualquer contratação.
- (C) Errada. Desenvolvimento baseado em componentes não decorre da existência de documentação formal.
- (D) Correta. Escopo estável, documentação formal e marcos sequenciais apontam para cascata/preditivo.
- (E) Errada. Kanban não é qualquer fluxo com etapas; será aprofundado em aula própria.

Erro provável: pegadinha FGV.

Questão 5 - Gabarito: A

- (A) Correta. Ciclo de vida trata as fases do software ao longo do tempo; modelo de processo organiza atividades.
- (B) Errada. Nenhum dos termos é linguagem de programação ou biblioteca.
- (C) Errada. Ciclo de vida não é necessariamente ágil, nem todo modelo é preditivo.
- (D) Errada. Reduz indevidamente conceitos amplos a codificação e teste.
- (E) Errada. Versionamento e execução são ferramentas/ambiente, não ciclo de vida.

Erro provável: confusão entre termos.

Questão 6 - Gabarito: C

- (A) Errada. Contratação e auditoria não representam as macroatividades clássicas do processo de software.

- (B) Errada. Programação e compilação são atividades técnicas, mas a lista ignora requisitos, validação e evolução.
- (C) Correta. Especificação, desenvolvimento, validação e evolução são macroatividades clássicas.
- (D) Errada. São atividades de banco de dados, não macroprocesso de software.
- (E) Errada. São conceitos de segurança/rede, não macroprocesso de desenvolvimento.

Erro provável: memorização.

Questão 7 - Gabarito: B

- (A) Errada. Ambas podem envolver diferentes atores; compilador não valida necessidade de negócio.
- (B) Correta. Verificação: construir corretamente o produto especificado; validação: construir o produto certo para a necessidade.
- (C) Errada. Usabilidade e controle de versão são temas distintos.
- (D) Errada. A distinção não depende exclusivamente de momentos fixos do ciclo.
- (E) Errada. CI/CD não substitui os conceitos de verificação e validação.

Erro provável: confusão entre termos.

Questão 8 - Gabarito: E

- (A) Errada. O cenário explicita relação com testes.
- (B) Errada. Espiral destaca riscos, não correspondência direta requisito-teste.
- (C) Errada. Incremental destaca entregas parciais.
- (D) Errada. Prototipação destaca exploração/aprendizado.
- (E) Correta. O V-Model evidencia correspondências entre especificação/projeto e níveis de teste.

Erro provável: conceito.

Questão 9 - Gabarito: D

- (A) Errada. Há entregas intermediárias funcionais.
- (B) Errada. As entregas devem ser utilizáveis, não descartadas.
- (C) Errada. O foco não é teste unitário.
- (D) Correta. Entregar partes funcionais sucessivas é característica do desenvolvimento incremental.
- (E) Errada. Modelo de maturidade avalia capacidade/processo, não descreve esse fluxo de entregas.

Erro provável: conceito.

Questão 10 - Gabarito: B

- (A) Errada. Normalização é tema de banco de dados.
- (B) Correta. Refinar a mesma funcionalidade por ciclos de feedback caracteriza iteração.
- (C) Errada. Controle de concorrência pertence a banco/sistemas concorrentes.
- (D) Errada. Assinatura digital é segurança.
- (E) Errada. Particionamento físico é implementação de dados/infraestrutura.

Erro provável: leitura apressada.

Questão 11 - Gabarito: A

- (A) Correta. Protótipos reduzem incerteza e podem ser descartáveis ou evolutivos.
- (B) Errada. Protótipo não é necessariamente produto final e pode exigir reengenharia.
- (C) Errada. Prototipação geralmente aumenta feedback dos usuários.
- (D) Errada. Protótipo descartável e incremento não são sinônimos.
- (E) Errada. Prototipação não elimina validação, documentação suficiente ou testes.

Erro provável: conceito.

Questão 12 - Gabarito: C

- (A) Errada. Cascata não destaca ciclos de mitigação de riscos.
- (B) Errada. V-Model foca correspondência com testes, não riscos como eixo central.
- (C) Correta. Espiral é clássico para desenvolvimento orientado a riscos.
- (D) Errada. Big Bang indica baixa disciplina, incompatível com criticidade.
- (E) Errada. Cópias físicas não definem processo de software.

Erro provável: pegadinha FGV.

Questão 13 - Gabarito: E

- (A) Errada. Troca incremental por cascata no primeiro item.
- (B) Errada. Inverte espiral e prototipação nos itens finais.
- (C) Errada. Começa por prototipação, mas a primeira descrição é incremental.
- (D) Errada. Não corresponde às descrições.
- (E) Correta. Entregas partes: incremental; fases sequenciais: cascata; riscos: espiral; versão parcial para aprender: prototipação.

Erro provável: memorização.

Questão 14 - Gabarito: E

- (A) Errada. Reuso não elimina testes; integração e compatibilidade precisam ser verificadas.
- (B) Errada. Reuso frequentemente envolve APIs e frameworks.
- (C) Errada. Aplica-se amplamente a sistemas corporativos.
- (D) Errada. Reuso não transforma sistema em protótipo descartável.
- (E) Correta. A alternativa apresenta vantagens e limites de forma equilibrada.

Erro provável: conceito.

Questão 15 - Gabarito: B

- (A) Correta, portanto não é a incorreta. Processos podem ser adaptados.
- (B) Incorreta e gabarito. Modernidade de processo não elimina requisitos, testes e comunicação.
- (C) Correta. Requisitos estáveis e previsibilidade favorecem abordagens preditivas.
- (D) Correta. Mudança e feedback favorecem adaptatividade.
- (E) Correta. Abordagens híbridas são plausíveis em ambientes públicos/corporativos.

Erro provável: exceção/incorreta.

Questão 16 - Gabarito: C

- (A) Errada. Inclui a III, que exagera o ágil.
- (B) Errada. Inclui a III e exclui a II.
- (C) Correta. I e II descrevem adequadamente preditivo e adaptativo.
- (D) Errada. A II está correta, mas a I também.
- (E) Errada. A III está errada.

Erro provável: pegadinha FGV.

Questão 17 - Gabarito: A

- (A) Correta. Agilidade não significa ausência de planejamento ou documentação útil.
- (B) Errada. Abordagens ágeis não proíbem registro escrito.
- (C) Errada. Ágil é compatível com testes automatizados e arquitetura evolutiva.
- (D) Errada. Planejamento também é técnico e de gestão do desenvolvimento.
- (E) Errada. Linguagem de programação não define essa regra.

Erro provável: conceito.

Questão 18 - Gabarito: E

- (A) Errada. Big Bang é o oposto de controle e validação.
- (B) Errada. Auditoria e iteração podem coexistir em abordagem híbrida.
- (C) Errada. O cenário não é necessariamente prototipação descartável.
- (D) Errada. Ferramenta de versionamento não substitui processo.
- (E) Correta. Combinar governança formal com ciclos de entrega e validação caracteriza abordagem híbrida.

Erro provável: leitura de cenário.

Questão 19 - Gabarito: D

- (A) Errada. Ferramentas não garantem maturidade nem processo completo.
- (B) Errada. Ferramentas apoiam, mas não substituem atividades essenciais.
- (C) Errada. Automação de build é prática de apoio, não dispensa processo.
- (D) Correta. Ferramentas são meios; processo define atividades, responsabilidades, critérios e artefatos.
- (E) Errada. Git é controle de versão; não define o V-Model.

Erro provável: confusão entre termos.

Questão 20 - Gabarito: B

- (A) Errada. Prototipação descartável aprende e descarta; cascata é sequencial.
- (B) Correta. Partes funcionais sucessivas: incremental; ciclos de refinamento: iterativo.
- (C) Errada. Espiral enfatiza risco, V-Model enfatiza testes correspondentes.
- (D) Errada. DevOps e waterfall não correspondem à distinção descrita.
- (E) Errada. Modelo físico/conceitual é assunto de modelagem de dados, não processo de software.

Erro provável: conceito.

14. Checklist final do que memorizar

Engenharia de software é mais ampla que programação: inclui requisitos, projeto, construção, testes, manutenção, qualidade, documentação e gestão.

Produto de software é resultado; processo de software é caminho; projeto é esforço temporário; sistema é mais amplo que software.

Macroatividades: especificação, desenvolvimento, validação e evolução.

Verificação: construí corretamente? Validação: construí o produto certo?

Cascata: fases sequenciais e previsibilidade em requisitos estáveis.

V-Model: correspondência entre especificação/projeto e níveis de teste.

Incremental: entrega partes funcionais; iterativo: refina por ciclos.

Prototipação: reduz incerteza; pode ser descartável ou evolutiva.

Espiral: ciclos orientados por riscos.

Reuso/componentes: reduz esforço quando adequado, mas não elimina testes e integração.

Preditivo não é automaticamente ruim; ágil não é ausência de disciplina. O que manda é a adequação ao cenário.

15. Fontes de calibração e referência

Fontes usadas para calibração de escopo, estilo e base técnica. As questões da aula são autorais e não copiam questões reais.

- Edital retificado TJSC 2026 - Analista de Sistemas - conteúdo de Engenharia de Software e Desenvolvimento.
- Manual Mestre de Calibração FGV para Aulas e Questões - fonte do projeto.
- FGV Conhecimento - Assembleia Legislativa do Estado de Goiás, Analista Legislativo - Analista de Sistemas, 2025: questões sobre metodologias prescritivas e ágeis.
- FGV Conhecimento - Ministério Público da União, Analista do MPU - Desenvolvimento de Sistemas, 2025: estilo de prova e itens contextualizados de desenvolvimento.
- FGV Conhecimento - Assembleia Legislativa do Estado do Amazonas, Analista Legislativo - Programador, 2025: questões contextualizadas sobre requisitos, sistemas e desenvolvimento.
- FGV Conhecimento - Assembleia Legislativa de Santa Catarina, Analista Legislativo III - Analista de Sistemas, 2024: questão sobre SWEBOK e conceitos de Engenharia de Software.
- IEEE Computer Society - SWEBOK - Software Engineering Body of Knowledge, usado como referência técnica geral de áreas de conhecimento de Engenharia de Software.
- AWS - Software Development Life Cycle (SDLC), usado apenas como apoio para definição contemporânea de ciclo de vida de desenvolvimento.
- Atlassian - Software Development Lifecycle, usado apenas como apoio secundário sobre etapas modernas do ciclo de vida.